

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA14 01

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO4: *Examine intermediate code generation techniques to enhance code optimization (BL4)*

Assessment Tool Weightage: 20%

Duration :60 Mins

On Class

QUESTIONS: 5

Total Marks:50

Annexure A

INDUSTRY PROBLEM- SOLVING TASK

Code of Conduct:

I Ismail M (192524128) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Ismail M (192524128)

Q1. Smart Retail Billing System

Marks: 10 Marks

Concepts Covered: Intermediate Languages, Declarations, Assignment Statements

Problem Statement: A retail store billing system calculates the final payable amount based on product price, quantity purchased, and discount percentage.

```
total_cost = price * quantity
discount_value = total_cost * discount
final_bill = total_cost - discount_value
```

(i) Write appropriate declarations for all variables with suitable data types.

Answer: Use floating-point types for monetary values and discount percentage, and an integer type for quantity.

```
float price;
int quantity;
float discount;
float total_cost;
float discount_value;
float final_bill;
```

If the implementation requires exact monetary arithmetic, a fixed-point/decimal representation can be preferred over binary floating point.

(ii) Generate the three-address intermediate code for the given statements.

Answer: Each operation is represented by one three-address instruction.

```
t1 = price * quantity
total_cost = t1
t2 = total_cost * discount
discount_value = t2
t3 = total_cost - discount_value
final_bill = t3
```

(iii) Represent the same using quadruples, triples, and indirect triples.

Answer: The following representations are equivalent.

Quadruples

No.	Operator	Arg1	Arg2 / Result
0	*	price	quantity → t1
1	=	t1	— → total_cost
2	*	total_cost	discount → t2
3	=	t2	— → discount_value
4	-	total_cost	discount_value → t3
5	=	t3	— → final_bill

Triples

```
0: (*, price, quantity)
1: (=, (0), total_cost)
2: (*, total_cost, discount)
3: (=, (2), discount_value)
4: (-, total_cost, discount_value)
5: (=, (4), final_bill)
```

Indirect Triples

Pointer table:

```
0 → Triple 0
1 → Triple 1
2 → Triple 2
3 → Triple 3
4 → Triple 4
5 → Triple 5
```

Triple table:

```
0: (*, price, quantity)
1: (=, (0), total_cost)
2: (*, total_cost, discount)
3: (=, (2), discount_value)
4: (-, total_cost, discount_value)
5: (=, (4), final_bill)
```

(iv) Identify possible optimization opportunities in the generated intermediate code.

Answer: The code contains temporary assignments such as `total_cost = t1` and `discount_value = t2`, followed by immediate use of those temporaries. These copies can be removed through copy propagation and temporary elimination. Common-subexpression elimination is not useful here because no identical expression is repeated. Algebraic simplification is also not applicable without changing the computation.

(v) Generate the optimized intermediate code and justify the improvements.

Answer: The unnecessary temporary assignments can be eliminated while preserving the required computation.

```
t1 = price * quantity
t2 = t1 * discount
t3 = t1 - t2
final_bill = t3
```

Improvement: the optimized form reduces the number of explicit assignment instructions and unnecessary temporary-to-variable copies. It preserves the original evaluation order and result. If the compiler permits direct assignment expressions, it can be represented even more compactly as:

```
total_cost = price * quantity
discount_value = total_cost * discount
final_bill = total_cost - discount_value
```

Q2. University Scholarship Eligibility System

Marks: 10 Marks

Concepts Covered: Boolean Expressions, Backpatching

Problem Statement: A student is eligible for a scholarship if $CGPA > 8.5$ AND (Family Income < 300000 OR Sports Quota = TRUE).

(i) Formulate the Boolean expression representing the eligibility criteria.

Answer: Let $C = (CGPA > 8.5)$, $I = (FamilyIncome < 300000)$, and $S = (SportsQuota == TRUE)$. Then the eligibility condition is:

$$\begin{aligned} ELIGIBLE &= C \text{ AND } (I \text{ OR } S) \\ &= (CGPA > 8.5) \text{ AND } ((FamilyIncome < 300000) \text{ OR } (SportsQuota == TRUE)) \end{aligned}$$

(ii) Generate intermediate code using short-circuit evaluation.

Answer: Short-circuit evaluation first checks CGPA. If it fails, the remaining conditions are not evaluated. If CGPA succeeds, family income is checked; if income fails, sports quota is checked.

```
1: if CGPA > 8.5 goto 2
2: if FamilyIncome < 300000 goto 5
3: if SportsQuota == TRUE goto 5
4: goto 6
5: goto 7
6: goto 7
7: SCHOLARSHIP_ELIGIBLE
```

(iii) Apply backpatching to resolve forward jumps.

Answer: For a clearer backpatching representation, use true and false lists. The true list of CGPA is patched to the start of the inner OR expression. The false list of the complete condition is patched to the failure label. The true list of the complete condition is patched to the scholarship action.

Initial symbolic form:

```
1: if CGPA > 8.5 goto _
2: goto _
3: if FamilyIncome < 300000 goto _
4: if SportsQuota == TRUE goto _
5: goto _
```

Backpatched form:

```
1: if CGPA > 8.5 goto 3
2: goto 6
3: if FamilyIncome < 300000 goto 7
4: if SportsQuota == TRUE goto 7
5: goto 6
6: NOT_ELIGIBLE
7: ELIGIBLE
```

Backpatching replaces unresolved forward-jump targets with the appropriate instruction labels after the target locations become known.

(iv) Construct the final labelled intermediate code.

Answer: The following labelled code directly implements the required short-circuit behavior.

```
L1: if CGPA > 8.5 goto L3
    goto L6
L3: if FamilyIncome < 300000 goto L7
    goto L4
L4: if SportsQuota == TRUE goto L7
    goto L6
L6: NOT_ELIGIBLE
    goto L8
L7: ELIGIBLE
L8: END
```

(v) Draw the corresponding control-flow graph.

Answer: The CFG consists of a decision node for CGPA, followed by the OR condition. Both successful paths of the OR condition reach ELIGIBLE, while failure reaches NOT_ELIGIBLE.

```
[L1] CGPA > 8.5?
   |True      |False
   v          v
[L3] Income < 300000? [L6] NOT_ELIGIBLE
   |True      |False
   v          v
[L7] [L4] SportsQuota == TRUE?
      |True    |False
      v        v
      [L7]    [L6]
      |
      v
      [L8] END
```

Q3. Smart Parking Management System

Marks: 10 Marks

Concepts Covered: Case Statements, Intermediate Languages

Problem Statement: A parking management system assigns parking zones based on vehicle type.

```
switch(vehicle_type)
{
    case 1: TWO_WHEELER;
    case 2: CAR;
    case 3: BUS;
    case 4: EMERGENCY_VEHICLE;
}
```

(i) Convert the switch statement into three-address intermediate code using conditional jumps.

Answer: Assuming each case transfers control to its corresponding action and then exits the switch:

```
1: if vehicle_type == 1 goto L1
2: if vehicle_type == 2 goto L2
3: if vehicle_type == 3 goto L3
4: if vehicle_type == 4 goto L4
5: goto L5
L1: TWO_WHEELER
    goto L5
L2: CAR
    goto L5
L3: BUS
    goto L5
L4: EMERGENCY_VEHICLE
    goto L5
L5: END_SWITCH
```

(ii) Generate an equivalent jump-table implementation.

Answer: A jump table stores the target address for each valid case. The vehicle type is used as an index after range checking.

```
1: if vehicle_type < 1 goto Ldefault
2: if vehicle_type > 4 goto Ldefault
3: t1 = vehicle_type - 1
4: goto JT[t1]
```

```
JT[0] → L1
JT[1] → L2
JT[2] → L3
JT[3] → L4
```

```
L1: TWO_WHEELER
    goto L5
L2: CAR
    goto L5
L3: BUS
    goto L5
```

```

L4: EMERGENCY_VEHICLE
    goto L5
Ldefault: DEFAULT_ACTION
L5: END_SWITCH

```

(iii) Represent the intermediate code using quadruples.

Answer: A quadruple representation for the conditional-jump version is:

No.	Operator	Arg1	Arg2 / Result
1	if==	vehicle_type	1 → L1
2	if==	vehicle_type	2 → L2
3	if==	vehicle_type	3 → L3
4	if==	vehicle_type	4 → L4
5	goto	—	— → L5
6	action	TWO_WHEELER	—
7	goto	—	— → L5
8	action	CAR	—
9	goto	—	— → L5
10	action	BUS	—
11	goto	—	— → L5
12	action	EMERGENCY_VEHICLE	—
13	goto	—	— → L5

(iv) Compare conditional branching and jump-table implementations.

Answer: Conditional branching performs a sequence of comparisons, so the number of tests can grow with the number of cases. A jump table performs a range check, computes an index, and performs an indexed transfer of control. Therefore, conditional branching is simple and suitable for a small or sparse set of cases, whereas jump tables are generally more efficient for many dense integer case labels. Jump tables require additional table storage and careful bounds checking.

(v) Recommend the best approach for a large-scale parking system with justification.

Answer: For a large-scale system with many consecutive integer vehicle-type codes, a jump table is recommended because it provides near-constant-time dispatch after bounds checking and avoids many sequential comparisons. If the case values are sparse or highly irregular, a compiler may instead prefer a decision tree or another dispatch structure.

Q4. Online Food Delivery Processing System

Marks: 10 Marks

Concepts Covered: Procedure Calls, Activation Records, Intermediate Code

Problem Statement: An online food delivery platform executes:

```
amount = calculateBill(order)
applyCoupon(amount)
generateInvoice(amount)
```

(i) Generate intermediate code for the procedure calls.

Answer: Use parameter passing, call, and return-value instructions.

```
1: param order
2: t1 = call calculateBill, 1
3: amount = t1
4: param amount
5: call applyCoupon, 1
6: param amount
7: call generateInvoice, 1
8: end
```

(ii) Illustrate parameter passing and return value handling.

Answer: For calculateBill, order is passed as an argument and the returned bill amount is stored in a temporary and then assigned to amount. The next procedures receive amount as a parameter. If applyCoupon modifies and returns a new amount, the code can capture that return value as well.

```
param order
ret = call calculateBill, 1
amount = ret

param amount
call applyCoupon, 1

param amount
call generateInvoice, 1
```

(iii) Design the activation record (stack frame) for each procedure call.

Answer: A generic activation record contains parameters, return address, saved control information, local variables, temporaries, and the return-value area. The exact layout is implementation-dependent.

```
Activation Record: calculateBill
+-----+
| Return value / result |
| Local variables      |
| Temporaries          |
| Saved registers/control |
| Return address       |
| Parameter: order     |
+-----+
```

Activation Record: applyCoupon

```
+-----+
| Return value (if any) |
| Local variables      |
| Temporaries          |
| Saved registers/control |
| Return address       |
| Parameter: amount    |
+-----+
```

Activation Record: generateInvoice

```
+-----+
| Return value (if any) |
| Local variables      |
| Temporaries          |
| Saved registers/control |
| Return address       |
| Parameter: amount    |
+-----+
```

(iv) Show the sequence of call and return operations using stack diagrams.

Answer: The caller first invokes calculateBill. Its activation record is pushed. After return, that frame is removed and amount is available. Then applyCoupon is called, followed by generateInvoice.

Before call:

TOP → Caller Frame

Call calculateBill:

TOP → calculateBill Frame

Caller Frame

Return:

TOP → Caller Frame

amount = returned value

Call applyCoupon:

TOP → applyCoupon Frame

Caller Frame

Return:

TOP → Caller Frame

Call generateInvoice:

TOP → generateInvoice Frame

Caller Frame

Return:

TOP → Caller Frame

(v) Explain how local variables, parameters, and return addresses are stored in the activation record.

Answer: Parameters are placed in the parameter area or passed through registers according to the calling convention. Local variables occupy compiler-assigned stack slots or registers. Temporaries may also be stored in the frame. The return address identifies the instruction in the caller to which control must return after the

procedure finishes. Saved registers and dynamic/static links may also be included when required by the language implementation.

Q5. Smart Industrial Safety Monitoring System

Marks: 10 Marks

Concepts Covered: Assignment Statements, Boolean Expressions, Backpatching, Procedure Calls

Problem Statement: An industrial safety system activates an emergency alarm if Temperature > 100 AND (Pressure > 80 OR GasLeak = TRUE). When satisfied, it invokes activateAlarm() and sendNotification().

(i) Write the Boolean expression representing the safety condition.

Answer: Let T = (Temperature > 100), P = (Pressure > 80), and G = (GasLeak == TRUE). The safety condition is:

SAFETY = (Temperature > 100) AND ((Pressure > 80) OR (GasLeak == TRUE))

(ii) Generate intermediate code using short-circuit evaluation.

Answer: Temperature is checked first. If it fails, no pressure or gas-leak test is needed. If temperature succeeds, pressure is tested; only when pressure fails is GasLeak evaluated.

```
1: if Temperature > 100 goto L3
2: goto L6
L3: if Pressure > 80 goto L7
4: if GasLeak == TRUE goto L7
5: goto L6
L6: goto L8
L7: param —
    call activateAlarm, 0
    call sendNotification, 0
L8: END
```

(iii) Apply backpatching to resolve all jump targets.

Answer: The unresolved true and false branches are patched after the target labels are known. The true list of the complete condition is patched to the alarm-call block, while all false paths are patched to the end.

Symbolic:

```
1: if Temperature > 100 goto _
2: goto _
3: if Pressure > 80 goto _
4: if GasLeak == TRUE goto _
5: goto _
```

After backpatching:

```
1: if Temperature > 100 goto L3
2: goto L8
3: if Pressure > 80 goto L7
4: if GasLeak == TRUE goto L7
5: goto L8
L7: call activateAlarm, 0
```

```
    call sendNotification, 0
L8: END
```

(iv) Generate intermediate code for the procedure calls executed when the condition is true.

Answer: The procedures have no explicit parameters in the given problem, so the call count is zero.

```
L7: call activateAlarm, 0
    call sendNotification, 0
    goto L8
```

(v) Represent the complete control flow using labelled instructions and explain the execution sequence.

Answer: Complete labelled intermediate code:

```
L1: if Temperature > 100 goto L3
    goto L8
L3: if Pressure > 80 goto L7
    goto L4
L4: if GasLeak == TRUE goto L7
    goto L8
L7: call activateAlarm, 0
    call sendNotification, 0
    goto L8
L8: END
```

Execution sequence: The system first checks whether temperature exceeds 100. If not, execution goes directly to END. If yes, pressure is checked. If pressure exceeds 80, the alarm block is executed immediately. Otherwise, GasLeak is checked. If GasLeak is TRUE, the alarm block executes; otherwise, execution goes to END. When the alarm block is entered, activateAlarm() executes first, followed by sendNotification().

RUBRICS FOR CALCULATION

Numerical Problems					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation